
BTS SIO 2024–2026 — Option SLAM
MCCI Business School — Session 2026
Réalisation Professionnelle n°1

RAPPORT DE PROJET

SUPERCAR-WEB

Application web dynamique de gestion d'un concessionnaire automobile

Nazra Binti Abdallah

Prestataire : **MultiSys SSII**

Année académique 2024–2026

Organisation cliente : **SuperCar**

SOMMAIRE

INTRODUCTION	3
PREMIÈRE PARTIE — LA DÉMARCHE PROJET	4
I. Présentation du contexte	4
II. Objectifs et problématiques	4
III. Missions et plan de travail	5
IV. Outils et technologies	5
DEUXIÈME PARTIE — ANALYSE DU PROJET	6
I. Analyse de l'existant	6
II. Expression des besoins	6
III. Orientations de solution	7
IV. Justification de la solution retenue	7
TROISIÈME PARTIE — CONCEPTION FONCTIONNELLE	8
I. Architecture de la solution	8
II. Modèle de données	8
III. Processus métier	9
QUATRIÈME PARTIE — DÉVELOPPEMENT TECHNIQUE	10
I. Environnement de développement	10
II. Ce qui a été développé	10
III. Sécurité de l'application	11
CINQUIÈME PARTIE — TESTS ET MISE EN PRODUCTION	13
I. Tests fonctionnels et de sécurité	13
II. Déploiement et livraison	14
CONCLUSION	15

INTRODUCTION

Dans le cadre de notre formation BTS SIO option SLAM à MCCI Business School (Île Maurice), nous avons été amenés à réaliser deux projets professionnels simulant des situations réelles de développement informatique. Ce rapport présente la première de ces réalisations : le projet SUPERCAR-WEB.

Ce projet met en scène deux acteurs principaux. D'un côté, SuperCar, un concessionnaire automobile souhaitant moderniser sa présence digitale et informatiser ses activités. De l'autre, MultiSys, une SSII (Société de Services en Ingénierie Informatique) chargée de concevoir et de développer la solution applicative en réponse au cahier des charges du client.

En tant qu'étudiant intégré dans l'équipe de MultiSys, ma mission a consisté à concevoir et développer un site web dynamique complet pour SuperCar. L'application devait permettre aux visiteurs de consulter le catalogue de véhicules, de s'inscrire à des événements, de soumettre des demandes d'essai en ligne, et à l'équipe interne de gérer l'ensemble du contenu depuis un panneau d'administration sécurisé.

Ce rapport retrace l'ensemble de ma démarche : de la compréhension du contexte et des besoins du client jusqu'à la livraison d'une application fonctionnelle, en passant par les choix techniques effectués, les développements réalisés et les tests conduits pour valider la solution.

PREMIÈRE PARTIE : LA DÉMARCHE PROJET

I. Présentation du contexte

SuperCar est, à l'origine, une société mauricienne spécialisée dans la vente de voitures neuves et d'occasion de diverses marques, importées de pays comme le Japon, Singapour, l'Afrique du Sud, les États-Unis, l'Allemagne ou la France. Les véhicules sont stockés dans quatre entrepôts répartis sur l'île, tandis que le siège social regroupe l'ensemble des départements : administration, vente, finance et ressources humaines.

Le fonctionnement commercial de l'entreprise repose sur un modèle d'essai en présentiel : un client se rend au siège social, choisit un modèle, effectue un essai sur place, et si celui-ci le satisfait, un véhicule identique lui est livré depuis l'un des entrepôts. Le personnel de vente perçoit une commission sur chaque transaction conclue.

Face à la concurrence croissante et à l'évolution des comportements d'achat, SuperCar a pris la décision de créer son propre site web dynamique et d'informatiser ses activités. Elle a pour cela sollicité les services de MultiSys, la SSII prestataire, avec pour mission de concevoir et développer une solution web adaptée à ses besoins.

Dans le cadre pédagogique de MCCI Business School, ce contexte a été directement inspiré d'une situation réelle mauricienne pour constituer le scénario de la Réalisation Professionnelle n°1 du BTS SIO.

II. Objectifs et problématiques

L'objectif principal du projet est de donner à SuperCar une meilleure visibilité en ligne et de lui permettre de mieux réagir face à la concurrence. Plus concrètement, le site doit remplir plusieurs fonctions essentielles : faire connaître les voitures disponibles au catalogue, donner des conseils aux clients et présenter les services de l'entreprise, générer des contacts, organiser la relation entre clients et vendeurs, permettre la gestion des demandes d'essai en ligne, et offrir un espace d'administration du contenu du site.

En analysant ces besoins, plusieurs problématiques techniques ont émergé. Comment concevoir une interface à la fois élégante et fonctionnelle, cohérente avec l'image premium du concessionnaire ? Comment sécuriser une application accessible publiquement, tout en protégeant les données des utilisateurs et en empêchant les accès non autorisés ? Comment gérer efficacement un catalogue de véhicules dynamique avec filtres et pagination, sans dégradation des performances ? Enfin, comment permettre des interactions fluides sans rechargement de page notamment

pour les inscriptions aux événements tout en garantissant l'intégrité des données côté serveur ?

III. Missions et plan de travail

Ma mission sur ce projet a couvert l'intégralité du cycle de développement, de la phase d'analyse jusqu'à la livraison de l'application. J'ai travaillé seul, en étant responsable de chaque composant de la solution : conception de la base de données, développement front-end, développement back-end, sécurisation de l'application et tests.

Le travail a été organisé en plusieurs phases successives. La première phase a été consacrée à l'analyse du cahier des charges, à la modélisation de la base de données et au maquettage de l'interface. Ensuite, j'ai configuré l'environnement de développement local (WampServer) et créé la structure de base du projet selon une architecture MVC simplifiée. La troisième phase a porté sur le développement des pages publiques et du design, suivie du développement back-end avec PHP et PDO. La cinquième phase a été dédiée à la sécurisation complète de l'application. J'ai ensuite développé le panneau d'administration, avant de procéder aux tests finaux et à la mise en production sur AlwaysData.

Cette approche progressive m'a permis de valider chaque composant avant d'aborder le suivant, réduisant ainsi les risques de blocage en fin de projet.

IV. Outils et technologies

Pour répondre aux exigences du cahier des charges, j'ai retenu le stack technique suivante, conforme aux préconisations de l'établissement et aux exigences du projet.

Côté serveur, j'ai utilisé PHP 8 comme langage de programmation principal, avec WampServer 3.2 comme environnement local intégrant Apache, PHP et MySQL. La connexion à la base de données est assurée par PDO (PHP Data Objects) avec des requêtes préparées natives, garantissant la sécurité contre les injections SQL. MySQL 8 est le système de gestion de base de données, administré via phpMyAdmin.

Côté client, j'ai développé les interfaces en HTML5 et CSS3 pur, sans framework CSS externe, avec un JavaScript natif pour les interactions dynamiques et les appels AJAX via l'API `fetch()`. Ce choix délibéré de travailler sans Bootstrap m'a permis de produire un design entièrement sur mesure, cohérent avec l'identité visuelle premium souhaitée pour un concessionnaire de voitures de luxe. Pour l'édition du code, j'ai utilisé Visual Studio Code.

DEUXIÈME PARTIE : ANALYSE DU PROJET

I. Analyse de l'existant

Avant d'entamer le développement, j'ai cherché à comprendre la situation existante de SuperCar sur le plan digital. En l'absence d'un site web ou d'un outil informatique dédié, l'entreprise gérait l'ensemble de ses activités commerciales par des moyens traditionnels : appels téléphoniques, échanges par email, et gestion interne sur tableurs. Cette organisation présentait plusieurs limites importantes.

Les clients n'avaient aucun moyen de consulter le catalogue de véhicules disponibles sans se déplacer physiquement au siège social. Les demandes d'essai étaient collectées manuellement, sans traçabilité centralisée ni possibilité de suivi. La communication sur les événements organisés par le concessionnaire, salons, lancements de nouveaux modèles, journées portes ouvertes était limitée à un mailing de base sans inscription en ligne. Enfin, toute modification du catalogue ou du contenu informatif nécessitait une intervention technique directe.

Cette analyse a confirmé la pertinence du projet : SuperCar avait un réel besoin de modernisation digitale, et la solution à développer devait couvrir l'ensemble de ces lacunes opérationnelles.

II. Expression des besoins

Le cahier des charges fourni par SuperCar via MultiSys exprimait des besoins à la fois fonctionnels et non-fonctionnels. Sur le plan fonctionnel, le site devait proposer une galerie des voitures avec pagination dynamique, un affichage des marques représentées, un formulaire de demande d'essai pour les clients potentiels, un module événements permettant l'inscription en ligne, et un formulaire de contact.

Le cahier des charges précisait également que le site devait pouvoir être utilisé par les vendeurs depuis un smartphone lors des consultations clientèles, ce qui imposait une conception responsive. Un outil d'administration sécurisé était requis pour permettre à l'équipe interne de mettre à jour l'ensemble du contenu sans intervention technique.

Sur le plan technique, les exigences étaient claires : utilisation de PHP côté serveur, MySQL comme SGBD, contrôle des formulaires via AJAX ou JavaScript, intégration d'au minimum une procédure stockée et un trigger dans la base de données. L'application devait également être hébergée sur un serveur distant en ligne, en complément du fonctionnement local.

III. Orientations de solution

Trois orientations principales ont été envisagées pour répondre aux besoins identifiés. La première option consistait à utiliser un CMS clé en main comme WordPress. Cette approche aurait permis un développement rapide grâce aux nombreux plugins disponibles. Cependant, elle ne permettait pas de démontrer les compétences techniques requises par le BTS SIO SLAM, et la personnalisation profonde nécessaire pour les fonctionnalités métier spécifiques (procédure stockée, AJAX sur mesure) aurait été difficile à mettre en œuvre dans ce cadre.

La deuxième option envisagée était l'utilisation d'un framework PHP moderne comme Laravel ou Symfony. Ces outils offrent une structure robuste, des outils de sécurité intégrés et une meilleure maintenabilité à long terme. Mais la courbe d'apprentissage importante et les contraintes de temps du projet académique rendaient cette option risquée pour une réalisation individuelle dans les délais impartis.

La troisième option, celle qui a finalement été retenue, est le développement en PHP natif selon une architecture MVC simplifiée. Cette approche garantit une maîtrise totale du code, une transparence complète sur le fonctionnement de chaque composant, et une adéquation parfaite avec les objectifs pédagogiques du BTS SIO SLAM.

IV. Justification de la solution retenue

Le choix du PHP natif avec architecture MVC simplifiée, couplé à MySQL via PDO, s'est imposé pour plusieurs raisons complémentaires. D'abord, sur le plan pédagogique, cette solution permet de démontrer concrètement la maîtrise des compétences fondamentales du BTS SIO SLAM : développement web côté serveur, gestion de base de données relationnelle, implémentation de mécanismes de sécurité, et développement d'interfaces dynamiques.

Ensuite, travailler sans abstraction de framework oblige à comprendre et à implémenter chaque composant manuellement : gestion de la connexion PDO, écriture des requêtes SQL paramétrées, mise en place des protections contre les injections et le XSS, gestion des sessions. Ce niveau de maîtrise est précisément ce que l'épreuve E6 du BTS SIO cherche à évaluer.

Enfin, cette approche m'a permis d'intégrer des éléments avancés comme les procédures stockées, trigger, vues SQL, index d'optimisation qui témoignent d'une maturité technique dépassant le simple développement CRUD, tout en restant dans un cadre technologique accessible et parfaitement maîtrisé.

TROISIÈME PARTIE : CONCEPTION FONCTIONNELLE

I. Architecture de la solution

L'application SuperCar est structurée selon une architecture MVC simplifiée, déposée dans le répertoire `wamp/www/supercar/` et accessible depuis `http://localhost/supercar/` en développement local. L'organisation des fichiers sépare clairement les responsabilités entre les différentes couches de l'application.

Le répertoire `config/` contient les deux fichiers fondamentaux du projet : `db.php` qui gère la connexion PDO à la base de données MySQL, et `session.php` qui assure le démarrage des sessions et centralise l'ensemble des fonctions helpers utilisées partout dans l'application (`estConnecte()`, `estAdmin()`, `exigerConnexion()`, `exigerAdmin()`, `nettoyer()`, `formaterPrix()`, `flashMessage()`, `afficherFlash()`).

Le répertoire `includes/` regroupe les composants partagés par toutes les pages publiques : `header.php` et `footer.php` pour la mise en page commune, `connexion.php` et `inscription.php` pour le traitement des formulaires d'authentification, et `deconnexion.php` pour la destruction sécurisée de session. Le répertoire `client/` contient les pages publiques du site, organisées en sous-répertoires `pages/` (catalogue, détail véhicule, événements, essai, contact) et `traitement/` (scripts POST et AJAX). Enfin, le répertoire `admin/` regroupe l'ensemble du panneau d'administration, avec son propre système de navigation et ses fichiers `includes` dédiés.

Cette séparation claire des responsabilités facilite la maintenance et l'évolution future de l'application. Elle garantit également que le code de chaque composant reste focalisé sur une seule préoccupation.

II. Modèle de données

La base de données `supercar_db` est encodée en `utf8mb4` et utilise le moteur InnoDB pour permettre l'activation des contraintes de clés étrangères. Elle comprend sept tables principales.

La table `utilisateurs` gère les comptes clients et administrateurs. Elle stocke les informations personnelles (prénom, nom, email unique, téléphone), le mot de passe haché en BCrypt, le rôle (client ou admin via un champ ENUM) et les horodatages de création et de mise à jour. La table `marques` stocke les quatre maisons automobiles représentées : Rolls-Royce, Bugatti, Maserati et Porsche, avec leurs slogans respectifs. La table `voitures` constitue le cœur du catalogue avec les 16 véhicules, reliée à `marques` par une clé étrangère, et stockant le prix en roupies mauriciennes, les performances, le type de carburant, le type de véhicule (Berline,

SUV, Coupé, Cabriolet, Hypercar), le chemin de l'image et des indicateurs de disponibilité et de mise en vedette.

Les tables `evenements` et `inscriptions_evenements` gèrent le module événementiel, cette dernière permettant des inscriptions d'utilisateurs connectés ou anonymes. La table `demandes_essai` enregistre les demandes de test drive, obligatoirement liées à un utilisateur connecté et à un véhicule. Enfin, la table `contacts` stocke les messages reçus via le formulaire de contact.

En complément de ces tables, la base intègre quatre vues SQL (`v_catalogue`, `v_vedette`, `v_evenements_actifs`, `v_demandes_essai`) qui simplifient les requêtes récurrentes, une procédure stockée `sp_cree_demande_essai()` qui centralise les vérifications métier avant l'insertion d'une demande, un trigger `tg_demande_essai_update()` qui automatise la mise à jour du champ `updated_at`, et sept index d'optimisation sur les colonnes les plus fréquemment interrogées.

III. Processus métier

Trois processus métier principaux ont été modélisés et implémentés. Le premier est le processus d'authentification : l'utilisateur soumet ses identifiants, la fonction `nettoyer()` assainit les données POST, une requête PDO préparée recherche l'utilisateur par email, `password_verify()` compare le mot de passe soumis au hash BCrypt stocké, et en cas de succès la session est initialisée avec l'identifiant, le prénom et le rôle de l'utilisateur.

Le deuxième processus concerne la demande d'essai en ligne. L'accès à la page `essai.php` est conditionné par `exigerConnexion()`. Si le paramètre GET `voiture_id` est présent dans l'URL, le modèle et la marque du véhicule concerné sont automatiquement pré-sélectionnés dans le formulaire, et les coordonnées de l'utilisateur (prénom, nom, email, téléphone) sont pré-remplies depuis `$_SESSION`. Après soumission, le script `traitement_essai.php` appelle la procédure stockée via PDO. Celle-ci vérifie d'abord que le véhicule est disponible, puis qu'aucune demande en attente n'existe déjà pour ce véhicule et cet utilisateur. Si les deux conditions sont remplies, la demande est insérée et la procédure retourne le code 'OK'.

Le troisième processus est l'inscription à un événement via AJAX. Un clic sur le bouton d'inscription ouvre une popup modale JavaScript, pré-remplie si l'utilisateur est connecté. La soumission déclenche une requête `fetch()` vers `traitement_inscription_event.php`, qui traite l'inscription côté serveur et répond en JSON. Selon le résultat, le popup se ferme et un message de confirmation ou d'erreur s'affiche dynamiquement, sans rechargement de page.

QUATRIÈME PARTIE — DÉVELOPPEMENT TECHNIQUE

I. Environnement de développement

L'environnement de développement local a été mis en place avec WampServer 3.2, qui intègre Apache 2.4, PHP 8 et MySQL 8 dans une suite unifiée. Cet environnement permet de simuler un serveur de production complet sur une machine Windows, avec la possibilité de configurer finement PHP (affichage des erreurs en développement, limites mémoire) et de gérer plusieurs projets via l'interface WampServer.

La base de données a été conçue et gérée avec phpMyAdmin, l'interface graphique intégrée à WampServer. Cet outil a facilité la création des tables, l'exécution des scripts SQL de création des vues et procédures stockées, ainsi que la vérification et la manipulation des données de démonstration.

Visual Studio Code a été utilisé comme éditeur de code principal et Google Chrome avec ses DevTools a servi pour visualiser le responsif.

II. Ce qui a été développé

Pages publiques et interface utilisateur

La page d'accueil (`index.php`) est construite autour de quatre sections. Le hero section affiche un fond sombre avec des halos de lumière créés par des radial-gradient positionnés en absolu, un titre en grand format et deux appels à l'action principaux. La section "À propos" présente une description de l'entreprise. La section événements affiche les titres des événements proposés, récupérés depuis la table `evenements` de la base de données. Enfin, la section marques présente un collage visuel des quatre maisons automobiles avec leurs identités graphiques distinctes.

Le catalogue (`voitures.php`) récupère et affiche les voitures groupées par marque depuis la base de données. Le CSS contient des styles pour des boutons de filtre (`.filter-btn`) dont la classe active utilise la couleur cyan (`--c1`).

La page détail (`voiture_detail.php`) affiche toutes les informations d'un véhicule avec un bouton « Réserver un essai » qui mène vers la page demande d'essai.

La page événements (`evenements.php`) liste l'ensemble des événements, passés et à venir, avec le module d'inscription modale AJAX décrit plus haut.

La page `essai.php` est protégée par `exigerConnexion()` et intègre la procédure stockée pour la création sécurisée des demandes. La page `contact` (`contact.php`) intègre un formulaire et une carte Google Maps en `iframe` pointant sur Ébène, Île Maurice.

Panneau d'administration

Le back-office, accessible depuis `/admin/`, propose huit modules distincts. Le tableau de bord (`admin/index.php`) affiche en temps réel les statistiques clés : nombre total de véhicules, demandes d'essai en attente, messages non lus et inscriptions aux événements. Les modules CRUD véhicules et événements permettent d'ajouter, modifier, supprimer des entrées et de gérer leur visibilité. Le module demandes d'essai liste les demandes avec la possibilité de changer leur statut de manière inline (en attente → confirmé → effectué → refusé). Les modules inscriptions, contacts, marques et utilisateurs complètent le dispositif d'administration.

Chaque fichier du répertoire `admin/` commence par l'appel à `exigerAdmin()`, garantissant qu'aucune page du back-office n'est accessible sans une session administrateur valide. La page `admin/login.php` est indépendante du login client et vérifie le champ rôle en base de données, empêchant tout compte client d'accéder au panneau.

III. Sécurité de l'application

La sécurité a été traitée comme une préoccupation transversale à toutes les couches de l'application, en se basant sur les 4 principes fondamentaux : Disponibilité, Intégrité, Confidentialité, Traçabilité.

Pour la confidentialité, les mots de passe ne sont jamais stockés en clair. L'algorithme BCrypt avec sel automatique est irréversible : `password_hash($mdp, PASSWORD_DEFAULT)` à l'inscription, et `password_verify($mdp_soumis, $hash_bdd)` à la connexion.

Pour l'intégrité et la disponibilité des données, toutes les requêtes SQL utilisent des paramètres liés séparément du code SQL via PDO, avec le mode `emulate_prepared = false` pour forcer les requêtes préparées natives. Il est ainsi impossible d'injecter du SQL quelle que soit la valeur des données soumises. La protection contre le XSS (Cross-Site Scripting) est assurée par deux mécanismes complémentaires. En entrée, la fonction centralisée `nettoyer()` applique `htmlspecialchars(trim($val), ENT_QUOTES, 'UTF-8')` sur toutes les données POST avant tout traitement. En sortie, `htmlspecialchars()` est systématiquement utilisée pour encoder toute donnée provenant de la base de données avant affichage.

Pour la traçabilité, la déconnexion effectue une destruction complète de session en trois étapes : vidage des variables session (`$_SESSION = []`), destruction de la session serveur (`session_destroy()`), et suppression du cookie client

(setcookie(session_name(), "", time()-3600)). Le trigger tg_demande_essai_update() garantit l'horodatage automatique de chaque modification d'une demande d'essai, même effectuée directement en base de données.

CINQUIÈME PARTIE — TESTS ET MISE EN PRODUCTION

I. Tests fonctionnels et de sécurité

Une série de tests a été conduite pour valider le comportement de l'application dans les scénarios d'utilisation nominaux et dans les cas d'erreur. Ces tests ont couvert l'ensemble des fonctionnalités développées.

Pour l'authentification, j'ai testé la connexion avec des identifiants valides, avec un email inexistant, avec un mot de passe incorrect, et la déconnexion complète. Le contrôle d'accès a été vérifié en tentant d'accéder directement à `essai.php` sans connexion (redirection vers le formulaire de connexion), aux pages `admin/` sans session (redirection vers `admin/login.php`), et avec un compte client sur les pages `admin` (refus explicite).

Le catalogue a été testé avec toutes les combinaisons possibles de filtres (marque seule, carburant seul, les deux combinés), ainsi que la navigation entre les pages de résultats. La demande d'essai a été testée en mode pré-rempli via le paramètre `voiture_id`, en tentant de soumettre une demande en doublon (la procédure stockée retourne un message d'erreur approprié), et en soumettant une demande valide avec vérification de l'enregistrement en base.

Pour les tests de sécurité, j'ai tenté des injections SQL dans tous les champs de formulaire et dans les paramètres GET — les requêtes préparées neutralisent systématiquement ces tentatives. Des balises `<script>` ont été injectées dans les champs texte pour tester la protection XSS — `htmlspecialchars()` encode correctement les caractères dangereux à l'affichage. L'ensemble de ces tests a été concluant.

II. Déploiement et livraison

Déploiement local

L'application SuperCar se déploie sur un environnement WampServer en copiant le répertoire supercar/ dans wamp/www/. La base de données est initialisée en exécutant le script SQL complet dans phpMyAdmin (création des tables, insertion des données de démonstration, création des vues, de la procédure stockée, du trigger et des index). La configuration PDO dans config/db.php est ensuite ajustée si nécessaire.

Hébergement en ligne

Conformément aux exigences du cahier des charges, l'application a également été déployée sur un serveur distant via la plateforme AlwaysData.net mise à disposition par l'établissement. Le transfert des fichiers a été effectué avec FileZilla, et la base de données a été recréeée en ligne via l'interface phpMyAdmin d'AlwaysData. Chaque étudiant dispose de ses propres identifiants d'accès, garantissant l'isolation des environnements.

Données de démonstration

La base de données est livrée avec un jeu de données complet permettant une démonstration immédiate de l'ensemble des fonctionnalités : 16 véhicules répartis sur 4 marques, 5 événements de démonstration de types variés (salon, lancement, portes ouvertes), et un compte administrateur préconfiguré (admin@supercar.mu / Admin@2024). Ce jeu de données a été conçu pour illustrer tous les cas d'usage de l'application et les différents statuts de demandes d'essai dans le back-office.

CONCLUSION

La réalisation du projet SUPERCAR-WEB a été une expérience complète et formatrice qui m'a permis de mettre en pratique l'ensemble des compétences acquises au cours de ma formation BTS SIO SLAM. Ce projet m'a confronté aux réalités concrètes du développement web professionnel : comprendre un cahier des charges client, faire des choix techniques justifiés, gérer seul l'intégralité d'un cycle de développement, et livrer une application fonctionnelle et sécurisée.

Sur le plan technique, ce projet m'a permis de maîtriser des concepts avancés en développement PHP : l'architecture MVC, l'utilisation de PDO avec requêtes préparées natives, la conception d'une base de données relationnelle avec des objets avancés (procédure stockée, trigger, vues, index), et l'implémentation de mécanismes de sécurité conformes au 4 principes fondamentaux de la sécurité des systèmes d'information : la **disponibilité**, l'**intégralité**, la **confidentialité** et la **traçabilité**. Le développement AJAX pour les inscriptions aux événements a été particulièrement enrichissant, me permettant de comprendre la communication asynchrone entre le client et le serveur et la gestion des réponses JSON.

L'aspect le plus challengeant du projet a été la conception d'une interface utilisateur premium sans framework CSS. Ce choix, volontairement exigeant, m'a obligé à maîtriser en profondeur CSS3 et ses possibilités notamment les gradients, les effets de glassmorphism et les transitions pour produire un design à la hauteur de l'image des marques représentées. Il m'a également démontré qu'il est tout à fait possible de réaliser des interfaces modernes et sophistiquées en CSS pur, sans dépendre d'un framework externe.

Ce projet m'a enfin appris la valeur de la rigueur dans l'organisation du travail. Planifier les phases, valider chaque composant avant de passer au suivant, et documenter le code au fur et à mesure sont des pratiques que ce projet m'a incité à adopter et qui se révèlent indispensables dans le développement logiciel professionnel.

Le site SuperCar répond à l'ensemble des besoins exprimés dans le cahier des charges : catalogue filtrable et paginé, gestion des événements avec inscription AJAX, demandes d'essai sécurisées via procédure stockée, formulaire de contact, et panneau d'administration complet avec contrôle d'accès par rôles. Cette réalisation constitue pour moi un aboutissement significatif de deux années de formation, et une base solide pour aborder le monde du développement web professionnel.